Model files:
e-learning-platform/src/app/models/

course.ts:

```
// src/app/models/course.ts
export interface Course {
  id: number;
  title: string;
  instructorId: number;
  domain: string;
  level: string;
  durationHrs?: number;
  tags?: string[];
  description?: string;

  // optional fields used for student UI
  price?: number;
  rating?: number;
  studentsCount?: number;

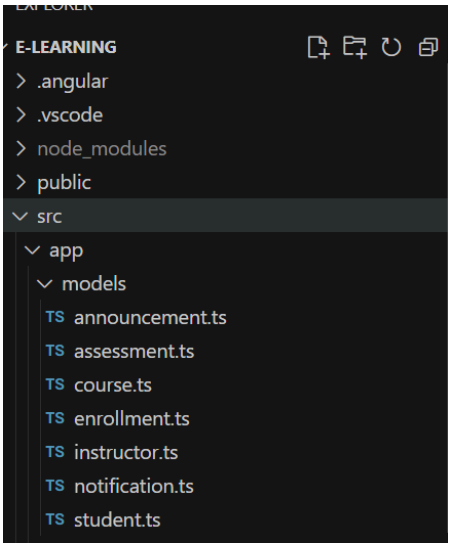
  // media fields (optional)
  thumbnail?: string;
  videoUrl?: string;
}
```

instructor.ts:

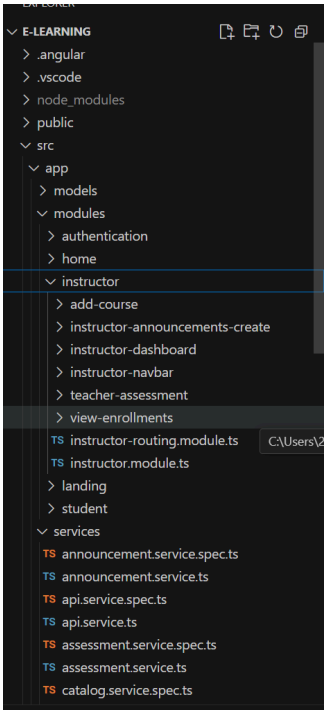
```
// src/app/models/instructor.ts
export interface Instructor {
  id: number|string;
  name: string;
  email: string;
}
```

enrollments.ts:

```
// src/app/models/enrollment.ts
export interface Enrollment {
  id?: number;
  studentId: number;
  courseId: number;
  enrollmentDate: string;
  progress: number;
  status: string;
}
```



Instructor Modules (components):
e-learning-platform/src/app/modules/instructor/



add-course/
Add-course.conponent.html:

```
<app-instructor-navbar></app-instructor-navbar>
<div class="container">
  <form [formGroup]="courseForm">
    <div class="card p-3">
```

```
      <h5 class="mb-3">{{ editingCourseId ? 'Edit Course' : 'Create
Course' }}</h5>

      <div class="row g-2">
        <div class="col-md-6">
          <label class="form-label">Title *</label>
          <input class="form-control" formControlName="title" />
        </div>
        <div class="col-md-6">
          <label class="form-label">Domain *</label>
          <input class="form-control" formControlName="domain" />
        </div>
      </div>

      <div class="row g-2 mt-2">
        <div class="col-md-6">
          <label class="form-label">Level *</label>
          <input class="form-control" formControlName="level" />
        </div>
        <div class="col-md-6">
          <label class="form-label">Duration (hrs)</label>
          <input class="form-control" type="number"
formControlName="durationHrs" />
        </div>
      </div>

      <div class="mt-2">
        <div class="col-md-6 mb-2">
          <label class="form-label">Tags</label>
          <input class="form-control" formControlName="tags"
placeholder="e.g. Angular,TypeScript" />
        </div>

        <div>
          <label class="form-label">Thumbnail URL</label>
          <input class="form-control" formControlName="thumbnail" />
        </div>
      </div>

      <div class="mt-2">
        <label class="form-label">Video URL</label>
        <input class="form-control" formControlName="videoUrl" />
      </div>
```

```
      <div class="mt-2">
        <label class="form-label">Description</label>
        <textarea class="form-control" rows="3"
formControlName="description"></textarea>
      </div>

      <div class="d-flex gap-2 mt-3">
        <button type="submit" class="btn btn-success"
(click)="OnSubmit()">Save Course</button>
        <button type="button" class="btn btn-outline-secondary"
(click)="courseForm.reset()">Reset</button>
      </div>

      <div *ngIf="message" class="mt-3">
        <div class="alert alert-info">{{ message }}</div>
      </div>
    </div>
  </form>

  <div class="col-md-4 mt-3">
    <!-- Quick list of existing courses -->
    <div class="card p-3">
      <h6 class="mb-2">Existing Courses</h6>
      <div *ngIf="courses?.length; else empty">
        <div *ngFor="let c of courses" class="mb-2">
          <div class="small fw-semibold">{{ c.title }}</div>
          <div class="small text-muted">{{ c.domain }} • {{ c.level
}}</div>
          <div class="mt-1">
            <button class="btn btn-sm btn-link"
(click)="edit(c)">Edit</button>
            <button class="btn btn-sm btn-link text-danger"
(click)="remove(c.id)">Delete</button>
          </div>
        </div>
      </div>
      <ng-template #empty>
        <div class="text-muted small">No courses yet</div>
      </ng-template>
    </div>
  </div>
</div>
```

add-coursr.component.ts:

```
// src/app/modules/instructor/add-course/add-course.component.ts
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { CourseService } from '../../../services/course.service';
import { CatalogService } from '../../../services/catalog.service';
import { Course } from '../../../models/course';

@Component({
  selector: 'app-add-course',
  standalone:false,
  templateUrl: './add-course.component.html',
  styleUrls: ['./add-course.component.css']
})
export class AddCourseComponent implements OnInit {
  courseForm!: FormGroup;
  message = '';
  courses: Course[] = [];
  editingCourseId: number | null = null;
  instructorId: number | null = null;

  constructor(
    private fb: FormBuilder,
    private courseService: CourseService,
    private catalog: CatalogService
  ) {}

  ngOnInit(): void {
    this.courseForm = this.fb.group({
      title: ['', Validators.required],
      instructorId: [null, Validators.required],
      domain: ['', Validators.required],
      level: ['', Validators.required],
      durationHrs: [null],
      tags: [''],
      description: [''],
      price: [null],
      rating: [null],
      studentsCount: [null],
      thumbnail: [''],
      videoUrl: ['']
```

```
    });

    const userRaw = localStorage.getItem('user');
    const user = userRaw ? JSON.parse(userRaw) : null;

    if (user && user.role === 'instructor') {

      if (user.instructorId) {
        this.instructorId = Number(user.instructorId);
        this.courseForm.patchValue({ instructorId: this.instructorId
});
        this.loadCourses(this.instructorId);
      } else {
        this.catalog.getInstructors().subscribe(list => {
          const found = list.find(i => (String(i.email) ||
'').toLowerCase() === (user.email || '').toLowerCase());
          if (found) {
            this.instructorId = Number(found.id);
            this.courseForm.patchValue({ instructorId:
this.instructorId });

            user.instructorId = this.instructorId;
            localStorage.setItem('user', JSON.stringify(user));
            this.loadCourses(this.instructorId);
          } else {
            // fallback: load all courses
            this.loadCourses();
          }
        }, err => {
          console.error('Failed load instructors', err);
          this.loadCourses();
        });
      }
    } else {
      // not logged-in as instructor -> just load all (admin dev mode)
      this.loadCourses();
    }
  }

  loadCourses(instructorId?: number) {
    if (instructorId) {
      this.courseService.getCourses({ instructorId }).subscribe({
        next: (data) => (this.courses = data),
```

```
        error: (err) => console.error(err)
      });
    } else {
      this.courseService.getCourses().subscribe({
        next: (data) => (this.courses = data),
        error: (err) => console.error(err)
      });
    }
  }

  OnSubmit() {
    if (!this.courseForm.valid) {
      this.message = 'Please fill required fields.';
      return;
    }

    const fv = this.courseForm.value;
    const tagsArray: string[] = fv.tags
      ? fv.tags.split(',').map((t: string) => t.trim()).filter((t:
string) => t.length > 0)
      : [];

    // ensure we use resolved instructorId if available
    const finalInstructorId = this.instructorId ?? (fv.instructorId ?
+fv.instructorId : undefined);

    const courseData: Omit<Course, 'id'> = {
      title: fv.title,
      instructorId: Number(finalInstructorId),
      domain: fv.domain,
      level: fv.level,
      // durationHrs: fv.durationHrs ? +fv.durationHrs : undefined,
      // tags: tagsArray,
      description: fv.description,
      price: fv.price ? +fv.price : undefined,
      rating: fv.rating ? +fv.rating : undefined,
      studentsCount: fv.studentsCount ? +fv.studentsCount : undefined,
      thumbnail: fv.thumbnail,
      videoUrl: fv.videoUrl
    };

    if (this.editingCourseId) {
```

```
      this.courseService.updateCourse(this.editingCourseId,
courseData).subscribe({
        next: () => {
          this.message = ' Course updated successfully';
          this.courseForm.reset();
          this.editingCourseId = null;
          this.loadCourses(this.instructorId ?? undefined);
        },
        error: (err) => {
          console.error(err);
          this.message = ' Failed to update course';
        }
      });
    } else {
      this.courseService.addCourse(courseData as Course).subscribe({
        next: () => {
          this.message = ' Course added successfully';
          this.courseForm.reset();
          this.loadCourses(this.instructorId ?? undefined);
        },
        error: (err) => {
          console.error(err);
          this.message = ' Failed to add course';
        }
      });
    }
  }

  // delete course
  remove(courseId: number | string) {
    const id = Number(courseId);
    this.courseService.deleteCourse(id).subscribe({
      next: () => {
        this.message = '🗑️ Course deleted successfully';
        this.loadCourses(this.instructorId ?? undefined);
      },
      error: (err) => {
        console.error(err);
        this.message = ' Failed to delete course';
      }
    });
  }
```

```
  edit(course: Course) {
    // make sure we only allow editing instructor's own courses (we're
already only loading theirs)
    this.editingCourseId = Number(course.id);
    this.courseForm.patchValue({
      title: course.title,
      instructorId: course.instructorId,
      domain: course.domain,
      level: course.level,
      durationHrs: course.durationHrs,
      tags: course.tags?.join(', '),
      description: course.description,
      price: course.price,
      rating: course.rating,
      studentsCount: course.studentsCount,
      thumbnail: course.thumbnail,
      videoUrl: course.videoUrl
    });
  }
}
```

instructor-dashboard/instructor-dashboard.html:

```
<app-instructor-navbar></app-instructor-navbar>

<div class="container mt-4">
  <!-- Header -->
  <div class="d-flex justify-content-between align-items-center mb-4">
    <div>
      <h2 class="fw-bold">Welcome back, Instructor!</h2>
      <p class="text-muted">Manage your courses and track
performance</p>
    </div>
    <div class="d-flex justify-content-end gap-4">
    <button class="btn btn-primary px-4" routerLink="add-assessment">+
Create Assessment</button>
        <button class="btn btn-primary px-4"
routerLink="announcements/create" >+ Create Announcement</button>

    </div>

  </div>
```

```
  <!-- My Courses -->
  <div>
    <h4 class="fw-bold mb-3">My Courses</h4>
    <div class="row g-3">
      <div class="col-md-4" *ngFor="let course of courses">
        <div class="card shadow-sm border-0 h-100">
          <img *ngIf="course.thumbnail" [src]="course.thumbnail"
class="card-img-top" alt="thumbnail" />
          <div class="card-body">
            <span class="badge bg-success mb-2" *ngIf="course.level ===
'Beginner'">{{ course.level }}</span>
            <span class="badge bg-danger mb-2" *ngIf="course.level ===
'Advanced'">{{ course.level }}</span>
            <span class="badge bg-warning text-dark mb-2"
*ngIf="course.level === 'Intermediate'">{{ course.level }}</span>

            <h5 class="fw-bold">{{ course.title }}</h5>
            <p>{{ course.description }}</p>
            <small class="text-muted">Duration : {{ course.durationHrs
}} hours</small>

            <div class="mt-3">
              <button class="btn btn-outline-secondary btn-sm
me-2">Edit</button>
              <button class="btn btn-primary btn-sm">View</button>
            </div>
          </div>
        </div>
      </div>
    </div>

    <div *ngIf="courses.length === 0" class="alert alert-info mt-3">
      You haven't created any courses yet.
    </div>
  </div>
</div>
```

instructor-dashboard.ts:

```
import { Component, OnInit } from '@angular/core';
import { CourseService } from '../../../services/course.service';
import { CatalogService } from '../../../services/catalog.service';
```

```
import { Course } from '../../../models/course';

@Component({
  selector: 'app-instructor-dashboard',
  standalone: false,
  templateUrl: './instructor-dashboard.component.html',
  styleUrls: ['./instructor-dashboard.component.css']
})
export class InstructorDashboardComponent implements OnInit {
  courses: Course[] = [];
  instructorId: number | null = null;

  constructor(private courseService: CourseService, private catalog:
CatalogService) { }

  ngOnInit(): void {
    const userRaw = localStorage.getItem('user');
    const user = userRaw ? JSON.parse(userRaw) : null;

    if (user && user.role === 'instructor') {
      if (user.instructorId) {
        this.instructorId = Number(user.instructorId);
        this.loadCourses();
      } else {
        // map user -> instructor record via email
        this.catalog.getInstructors().subscribe(list => {
          const found = list.find(i => (String(i.email) ||
'').toLowerCase() === (user.email || '').toLowerCase());
          if (found) {
            this.instructorId = Number(found.id);
            user.instructorId = this.instructorId;
            localStorage.setItem('user', JSON.stringify(user));
          }
          this.loadCourses();
        }, err => {
          console.error('Failed to load instructors', err);
          this.loadCourses();
        });
      }
    } else {
      // not an instructor (for dev) -> load nothing or all
      this.loadCourses();
    }
```

```
  }

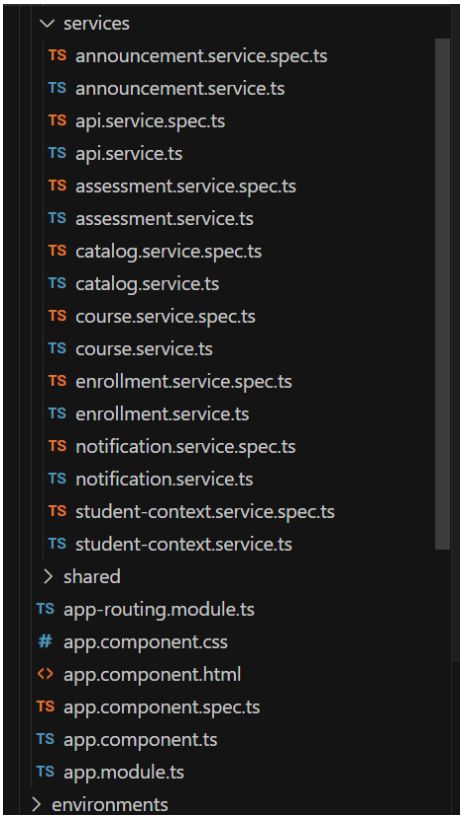
  loadCourses() {
    if (this.instructorId) {
      this.courseService.getCourses({ instructorId: this.instructorId
}).subscribe({
        next: (data) => (this.courses = data),
        error: (err) => console.error(err)
      });
    } else {
      // fallback: load all courses (or none). I keep load-all so dev
can still see something.
      this.courseService.getCourses().subscribe({
        next: (data) => (this.courses = data),
        error: (err) => console.error(err)
      });
    }
  }
}
```

instructor.module.ts:

```
// src/app/modules/instructor/instructor.module.ts
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule, ReactiveFormsModule } from '@angular/forms';
import { RouterModule } from '@angular/router';
import { InstructorRoutingModule } from './instructor-routing.module';
import { InstructorDashboardComponent } from
'./instructor-dashboard/instructor-dashboard.component';
import { AddCourseComponent } from './add-course/add-course.component';
import { ViewEnrollmentsComponent } from
'./view-enrollments/view-enrollments.component';
import { SharedModule } from "../../shared/shared.module";
import { InstructorAnnouncementsCreateComponent } from
'./instructor-announcements-create/instructor-announcements-create.comp
onent';
import { InstructorNavbarComponent } from
'./instructor-navbar/instructor-navbar.component';
```

```
@NgModule({
  declarations: [
    InstructorDashboardComponent,
    AddCourseComponent,
    ViewEnrollmentsComponent,
    InstructorAnnouncementsCreateComponent,
    InstructorNavbarComponent
    ],
  imports: [
    CommonModule,
    FormsModule,
    ReactiveFormsModule,
    RouterModule,
    InstructorRoutingModule,
    SharedModule
]
})
export class InstructorModule {}
```

Services:



app/serviecs/
api.services.ts:

```
import { Injectable } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { environment } from '../../environments/environment';
import { Observable } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class ApiService {
  private base = environment.apiBase;
  constructor(private http: HttpClient) {}

  get<T>(path: string, params?: Record<string, any>): Observable<T> {
    let p = new HttpParams();
    if (params) {
      Object.entries(params).forEach(([k, v]) => {
        if (v !== undefined && v !== null && v !== '') p = p.set(k,
String(v));
      });
    }
    return this.http.get<T>(`${this.base}/${path}`, { params: p });
  }

  post<T>(path: string, body: any): Observable<T> {
    return this.http.post<T>(`${this.base}/${path}`, body);
  }

  put<T>(path: string, body: any): Observable<T> {
    return this.http.put<T>(`${this.base}/${path}`, body);
  }

  delete<T>(path: string): Observable<T> {
    return this.http.delete<T>(`${this.base}/${path}`);
  }
}
```

catalog.service.ts:

```
import { Injectable } from '@angular/core';
import { ApiService } from './api.service';
import { Course } from '../models/course';
import { Student } from '../models/student';
import { Instructor } from '../models/instructor';
```

```
import { Observable } from 'rxjs';
import { map } from 'rxjs/operators';

@Injectable({ providedIn: 'root' })
export class CatalogService {
  constructor(private api: ApiService) {}

  getCourses(filter?: { search?: string }): Observable<Course[]> {
    return this.api.get<Course[]>('courses').pipe(
      map(courses => {
        if (filter?.search) {
          const term = filter.search.toLowerCase();
          return courses.filter(c =>
            c.title.toLowerCase().includes(term) ||
            (c.tags && c.tags.some(tag =>
tag.toLowerCase().includes(term)))
          );
        }
        return courses;
      })
    );
  }

  getCourse(id: number) { return this.api.get<Course>(`courses/${id}`);
}
  createCourse(c: Course) { return this.api.post<Course>('courses', c);
}
  updateCourse(c: Course) { return
this.api.put<Course>(`courses/${c.id}`, c); }
  deleteCourse(id: number) { return
this.api.delete<void>(`courses/${id}`); }

  getStudents() { console.log( this.api.get<Student[]>('students'));
    return this.api.get<Student[]>('students'); }
  getInstructors() { return this.api.get<Instructor[]>('instructors');
}
}
```

course.service.ts:

```
import { Injectable } from '@angular/core';
import { ApiService } from './api.service';
```

```
import { Course } from '../models/course';
import { Observable } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class CourseService {
  constructor(private api: ApiService) {}

  addCourse(course: Course): Observable<Course> {
    return this.api.post<Course>('courses', course);
  }

  getCourses(filter?: Record<string, any>): Observable<Course[]> {
    return this.api.get<Course[]>('courses', filter);
  }

  getCourseById(id: number): Observable<Course> {
    return this.api.get<Course>(`courses/${id}`);
  }

  updateCourse(id: number, course: Partial<Course>): Observable<Course>
{
    return this.api.put<Course>(`courses/${id}`, course);
  }

  deleteCourse(id: number): Observable<void> {
    return this.api.delete<void>(`courses/${id}`);
  }
}
```

Here is the github link of this project with path :
https://github.com/Rizwan1018/E-LEARNING-PLATFORM/tree/main/src/app/